

Tedlar Lead Agent — Design & Results

DuPont Tedlar, Graphics & Signage · lead discovery → qualification → outreach

1. Agent workflow

The brief invites a swarm of chatty agents. I built the opposite: a **deterministic pipeline of seven stages**, with the LLM confined to the two jobs it is genuinely better at than code — reading a messy web page, and writing prose over facts that are already established.

```
discover_events → extract_companies → dedupe → enrich_companies → qualify → find_contacts → draft_outreach → dashboard
```

Everything else is plain Python: scoring, deduplication, retry and backoff, persistence, filtering, provider selection. That choice is the whole architecture, and it buys three things a multi-agent design does not.

Reproducibility. The same inputs give the same score, every run. A sales manager can argue with the number because the number is a formula, not a mood.

Debuggability. When a lead looks wrong, the breakdown says which of five components produced the points and which URL backed each one. There is no "the agent decided" step to shrug at.

Cost. The LLM is called twice per qualified lead — once to read the site, once to write — instead of once per reasoning hop.

Orchestration is a sequential runner over a shared `RunContext`. The load-bearing primitive is `ctx.attempt(...)`: per-record work runs inside it, an *expected* failure (HTTP error, invalid model output, missing field) becomes a persisted `StageError` plus `status="incomplete"` on that record, and the run continues. A genuine programming error still propagates. **Partial success is the designed outcome; a crash is a bug.**

The LLM boundary is enforced, not just intended. Model output is parsed into a Pydantic model before it can touch the database; invalid output triggers exactly one repair round-trip carrying the validator's own error message, then the record degrades rather than the run failing. Both LLM stages have a deterministic fallback, so the entire pipeline runs end-to-end with no API key at all.

2. Data processing

Events. Six anchor institutions — ISA Sign Expo, PRINTING United Expo, FESPA Global Print Expo, the International Sign Association, PRINTING United Alliance, OAAA — are seeded as URLs and then *verified live*: fetched, title confirmed, source recorded. Anything unreachable is kept and marked unverified, never silently dropped. Search expansion then adds events we did not seed, gated by a title heuristic and one-event-per-host so the channel cannot flood the table with "Register" pages.

Companies arrive through three channels, because each fails differently:

1. *Exhibitor directory scrape.* Works on plain-HTML directories. The flagship shows front their lists with a JavaScript app over an API that rejects server-side clients — those record a miss in the error log.
2. *ICP-shaped search.* The channel that scales: retargeting is new query templates, not new code.
3. *A verified ICP roster* — names and public URLs only. Every fact still comes from fetching the site. This is the one hand-curated input and is deliberately the cheapest thing in the repo to delete.

Deduplication is deterministic: registrable domain first (`www.`, subdomains and multi-part public suffixes normalised), then normalised legal name with a 0.90 fuzzy threshold, so "Avery Dennison", "Avery Dennison Corp." and `graphics.averydennison.com` collapse into one record while Arlon and Avery do not. Field-wise merge keeps what we already know and fills the holes.

Enrichment follows the company's own navigation — ranking links by nav keywords rather than guessing `/about` — and turns the text into structured facts. The extraction prompt is null-preferring and forbidden from inferring: "a global leader" is explicitly *not* a size statement. A regex backstop reads revenue only when a revenue word sits within 60 characters of the figure, which is what stops "a \$2 billion market" being recorded as company turnover.

Provenance is structural. Every HTTP response is written to a content-addressed cache with URL, status and timestamp — failures included, so a cached replay reproduces the original run's error log rather than reporting cache misses. Every claim in the UI carries a clickable source URL, and that URL is the one the fact actually came from: a revenue figure read off an aggregator cites the aggregator, not the company's homepage, and is labelled a third-party estimate.

3. Qualification, contacts, outreach

The score is 0–100 across five weighted components — industry fit (30), application fit (25), size (15), event engagement (15), pain-point alignment (15) — with the ICP vocabulary, bands and thresholds all living as data in one file. Each component that scores emits an `Evidence` row carrying the URL it came from.

`confidence` is separate from score and measures how much of the score is URL-backed, penalised for a missing website, missing size data, or an unenriched record. This separation matters: a high-score/low-confidence lead is a research task, not a call.

The LLM writes the rationale from the finished breakdown, and two guards run afterwards — a cited URL outside the supplied evidence set flags the record, and any sentence making a numeric claim we cannot match to a known figure is removed.

Contacts sit behind a `ContactProvider` protocol with `find_contacts` and `is_configured`.

`PublicWebContactProvider` works today. `ClayProvider` and `SalesNavigatorProvider` are written in full — the real Clay table-webhook poll/callback flow and the real SNAP `salesApiLeadSearch` request shape — and gated on credentials. Scraping Sales Navigator would breach LinkedIn's terms, so without a partner token that provider contributes one honest artefact: a pre-filtered Sales Navigator deep link a licensed rep can click. Enabling Clay in production is `CONTACT_PROVIDERS=clay,public_web` plus two secrets, with no caller changes.

Outreach is drafted only for tier A/B leads that have a contact *and* evidence. The model must return the hook fact and the URL it came from; a validator rejects the draft if that URL is not one we supplied, if the body or subject runs long, or if it contains banned filler. Rejected drafts fall back to a deterministic template built from the same evidence, labelled as such in the UI.

4. Implementation results

Pipeline runs end to end in both modes, and a cached replay reproduces a live run exactly — same 54 companies, same 47 enrichments, same 13 handled errors — in 2.4 seconds against roughly fifteen minutes live. **87 tests, no network**, covering scoring bands and tier boundaries, deduplication, LLM-output repair and failure, cache/retry behaviour, provider fallback, outreach validation, and the resilience guarantee — a stage that raises does not lose prior work, and later stages still run.

The dashboard shows summary metrics, live stage progress, filters by event / tier / score / text, a lead table, and a detail panel with the score breakdown, rationale, clickable evidence, contact card with Sales Navigator link, and a copy/edit/approve outreach editor. A dedicated error tab lists every handled failure, because a pipeline that hides its failures is worse than one that has none.

What the constraints cost, stated plainly. MapYourShow-backed exhibitor directories are JavaScript-gated and their API refuses server-side clients — a headless browser is the fix. Without a paid search key, recall on category leaders is thin, which is why the roster channel exists. Most private companies never publish revenue, so the size component is frequently zero; the scorer treats that as a confidence penalty rather than inventing a band. Each of these is visible in the

error log rather than papered over — which is the behaviour I would want from this system in production, and the reason it was built this way.

5. Operating it: cost, speed, and knowing the keys work

Three things separate a prototype that demos from one a team could actually run.

Preflight. `uv run python -m app.preflight` makes the smallest real call every configured provider offers, reports what it found without printing a secret, and exits non-zero if a run would break. A live run costs minutes and credits, so failing fast on a bad key is worth its own command; `--strict` makes it a CI smoke test. It is also how the Apollo finding below surfaced in seconds rather than halfway through a run.

Targeted LLM spend. Extraction runs on Haiku; only tier A/B leads earn a Sonnet rationale. On a wide run most of the corpus is disqualified, and writing prose about a company scoring 12/100 spends reasoning tokens on something no rep will open — those still get a deterministic rationale from the same breakdown. Every run records token usage and estimated cost, and the dashboard turns it into **cost per qualified lead**, which is the number a GTM team budgets against. Prompt caching is deliberately absent: these system prompts sit well under the minimum cacheable prefix, so it would add complexity and cache nothing.

Bounded concurrency. Enrichment is almost entirely network wait — four or five fetches plus an LLM call per company, each against a different host — so it runs several companies at once rather than one at a time, with the fetcher's global cap and per-host delay still underneath. Parallelism happens across hosts; no single site sees a burst.

What integrating real providers actually taught us. Apollo's people-search endpoint is gated behind paid plans and returns a 403 on the free tier, while its organization endpoint is open — so Apollo became the firmographics source that lifted the size component off the floor, and decision-makers come from site-restricted LinkedIn search instead. That search only worked once the queries stopped asking for verbatim ICP titles ("VP Product Development") and started asking for the seniority words that actually appear in profile headlines. Both are the kind of thing no amount of design finds; only wiring up the real API does.

6. Scaling path

Bounded async throughout — a global concurrency semaphore plus a per-host lock and delay. Per-record isolation already exists in every stage, so moving to a task queue means enqueueing the same coroutines with no stage changes. The ICP is one file. `DATABASE_URL` is the only thing between SQLite and Postgres. Search and contact providers are protocols with registries, so a new vendor is a new file.